TRAFFIC LIGHT DETECTION

DA 213 - Python Programming Lab

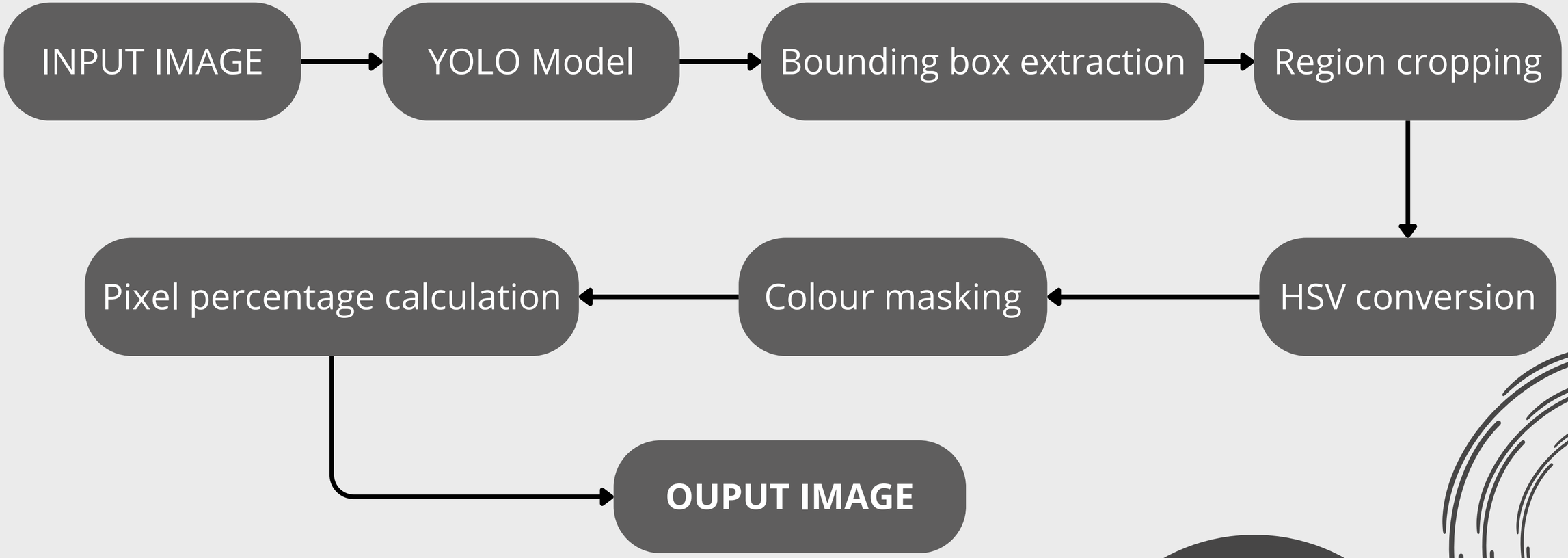
Team Members :
Pratyaksha Jha 240150025
Somita Agarwal 240150035
Sunkari Sharanya 240150036
Neeharika 240150008

INTRODUCTION

Problem Statement : Build a unified system capable of detecting traffic lights under both day and night conditions, ensuring robustness across varying environments.

Objectives : To build a reliable computer vision pipeline that can spot traffic lights in real-world images and detect its colour accurately. Our goal was to combine deep learning for the initial detection and image processing for colour classification.

SYSTEM OVERVIEW



METHODOLOGY

Step 1 : Traffic light detection

We used a **YOLO (You Only Look Once)** model from the Ultralytics framework to detect traffic lights in images.

Step 2 : Bounding Box Extraction

It also outputs the coordinates of the bounding boxes, which is used in further processing. Bounding boxes are constructed around the detected traffic lights and there is a label with the confidence score.

Step 3 : Region Cropping

The detected bounding box coordinates are used to extract the region containing the traffic light from the original image

METHODOLOGY

Step 4 : Colour detection using HSV

The image is converted from BGR to HSV to separate colour information from brightness and this works better in different lighting conditions

Step 5 : Pixel based classification

The number of pixels corresponding to each color is calculated. The percentage of each color is computed relative to the total detected pixels.

Step 6 : Output

The color with the highest percentage is selected as the final traffic light state. The annotated image is displayed with the bounding box and the detected percentage of each colour in the box.

DATASET DESCRIPTION

Dataset Source:

We took the data from Roboflow universe.

Dataset Features:

The dataset has a .txt file for each image with information about the coordinates of the bounding box and the class it belongs to. There are two classes - traffic_light and no_traffic_light. It initially contained 1677 images.

Dataset Augmentation:

We performed data augmentation to introduce variability and to make the dataset align more with real data. We specifically performed the following steps - add a blur of 4.8px , increase/decrease the brightness by 50%, and add noise by 2.75 %.

The final dataset contained **4031 images.**

IMPLEMENTATION DETAILS

YOLO (You Only Look Once) :

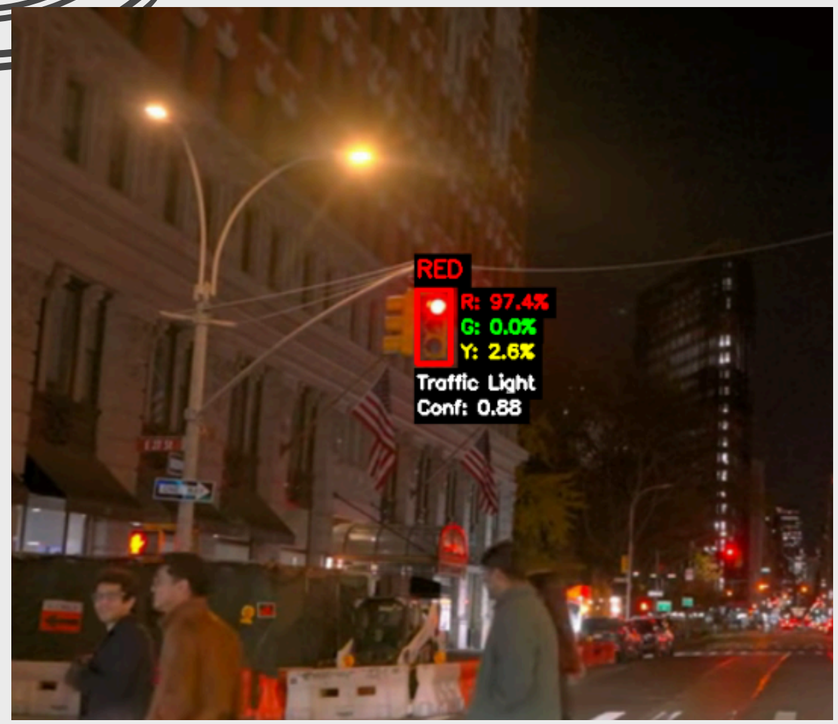
- Real-time, single-pass detection.
- Uses a **Convolutional Neural Network (CNN)** to identify complex visual patterns.
- Divides image into cells; each predicts:
  - Bounding Boxes
  - Confidence Scores
  - Class Probabilities
- Applies **Non-Maximum Suppression (NMS)** to remove overlapping duplicates.
- Our Implementation:
  - Labels: traffic_light & no_traffic_light
  - Passes isolated bounding box data to OpenCV for color processing.

IMPLEMENTATION DETAILS

OpenCV

- It is an open-source library used for image processing and computer vision applications
- Convert the **cropped traffic light into HSV colorspace** because **HSV separates color from brightness**, making detection more reliable under different lighting conditions
- Define HSV ranges for Red, Green and Yellow
- Create binary masks to isolate each color
- Count pixels in each mask
- Color with highest percentage as final traffic light state

RESULTS



Precision	96.7%
Recall	95.4%

mAP50	97.7%
mAP50-95	72.3%

WORKFLOW

User uploads an image

↓

User calls the function

detect_lights(“path-to-the-image”)

↓

The system runs the YOLO model and performs HSV color detection

↓

The system returns the image with bounding box around the traffic light along with the detected color

LIMITATIONS

1. Reduced accuracy in rainy night conditions
2. Limited detection range
3. High Computational cost

FUTURE SCOPE

1. Enhance image preprocessing
2. Expand dataset diversity
3. Advanced color classification
4. Real-time video processing

REFERENCES

1. Ultralytics YOLO GitHub
2. OpenCV Documentation
3. Roboflow Dataset Platform
4. Citations:

   Traffic light Detection using OpenCV and YOLO

   Traffic Light Detection and Recognition using YOLOv8

Traffic Light Color Identification for Automatic Traffic Light Violation Detection System